

CO_4 AT_2 CASE STUDY

NAME : Ranjitha. R

REG NO : 192421285

CASE STUDY SCENARIO

A multi-specialty hospital is planning to modernize its Hospital Management System (HMS) to efficiently manage different types of patients such as Inpatients, Outpatients, and Emergency Patients.

The current system has several issues:

- Redundant data storage for patient details
- Lack of flexibility in handling different billing mechanisms
- Difficulty in extending the system for new patient categories
- Inefficient handling of patient records in memory

To improve system performance and scalability, the hospital wants to redesign the system using advanced Object-Oriented Programming concepts in C++.

The system using advanced Object-Oriented Programming concepts in C++.

Case Study Task

You are required to analyze, design, implement, and evaluate a C++ application using:

- Inheritance (types of inheritance)
- Virtual base classes
- Abstract classes
- Constructors in derived classes
- Member classes (composition)
- Pointers and pointers to objects
- this pointer
- Pointer to derived classes
- Runtime polymorphism

Part A:

1. Identify the limitations of the existing system.
2. Explain how OOP principles can solve these issues.
3. List the functional requirements of the improved system.

Part B:

1. Design a class hierarchy:
 - o Abstract base class: Patient
 - o Derived classes:
 - InPatient
 - OutPatient
 - EmergencyPatient
2. Extend using multiple inheritance:
 - o Create CriticalPatient (inherits from InPatient and EmergencyPatient)
3. Use:
 - o Virtual base classes to resolve ambiguity
 - o A member class (Doctor) inside Patient

Part C:

1. Compare:
 - o Inheritance vs Composition in patient management
 - o Static vs Dynamic binding
2. Analyze:
 - o The role of virtual base classes in diamond problem
 - o Benefits of pointers in dynamic patient handling
3. Justify:
 - o Your design approach
 - o How it improves scalability and maintainability

Part D:
Develop a C++ program that:

1. Uses:
 - o Abstract class with pure virtual function calculateBill()
 - o Function overriding (polymorphism)
2. Implements billing logic:
 - o InPatient → Room charge + Treatment cost
 - o OutPatient → Consultation fee
 - o Emergency → Additional emergency charges
3. Demonstrates:
 - o Pointer to base class (Patient*)
 - o Dynamic allocation (new, delete)
 - o Use of this pointer
4. Includes:
 - o Menu-driven program using control structures

Sample Scenario

Input:

Patient Type: Critical Patient

Room Charge: 2000

Treatment Cost: 3000

Emergency Charge: 1500

Output:

Total Bill = ₹6500

1. Limitations of the Existing Hospital Management System

The existing hospital system may have the following limitations:

- Patient records may be maintained manually, causing data errors.
- Searching and updating patient information can be time-consuming.
- Doctor and patient information may not be properly organized.
- Emergency cases may not be prioritized efficiently.
- Billing and payment calculations may contain manual errors.
- Department and bed availability may be difficult to track.
- The system may not provide proper separation of data and operations.
- Reusing and maintaining the software can be difficult without proper object-oriented design.
- There may be limited security for sensitive patient information.
- Generating reports and accessing hospital information may take more time.

2. How OOP Principles Address These Limitations

OOP Principle	Application in Hospital System
Encapsulation	Patient, doctor, billing and department data are kept private and accessed through methods.
Abstraction	Only necessary operations such as registration, billing and appointment management are exposed to users.
Inheritance	Common properties can be inherited from a Person class by Patient, Doctor and Staff.
Polymorphism	Different types of users can implement common functions such as displayDetails() differently.
Classes and Objects	Real-world entities such as Patient, Doctor, Department and Bill can be represented as objects.
Data Hiding	Sensitive patient and billing information can be protected using private members.
Reusability	Common functionality can be reused through inheritance and functions.

3. Functional Requirements of the Proposed System

The proposed Hospital Management System should provide:

1. Patient Registration

- Add new patient details.
- Store patient ID, name, age, disease and department.

2. Patient Management

- View patient details.
- Search patient records.
- Update patient information.

3. Doctor Management

- Add doctor details.
- Store specialization and department.
- Display doctor information.

4. Appointment Management

- Schedule appointments between doctors and patients.
- View appointment details.

5. Emergency Management

- Identify emergency patients.
- Give priority to emergency cases.

6. Department Management

- Add hospital departments.
- Maintain department information.
- Track available beds and doctors.

7. Billing System

- Calculate consultation charges.
- Calculate treatment and medicine charges.
- Generate the total bill.

8. Record Management

- Store and retrieve hospital information efficiently.
- Search records using patient or doctor IDs.

9. Report Generation

- Display patient, doctor, department and billing information.

10. Secure Data Handling

- Protect sensitive patient and hospital information using encapsulation and access control.

CODE:

```
#include <iostream>
#include <vector>
#include <memory>
using namespace std;
class Person {
protected:
    int id;
    string name;
    int age;
public:
    Person(int i, string n, int a) : id(i), name(n), age(a) {}
    virtual void display() const = 0; // Pure virtual function
    int getId() const {
        return id;
    }
    virtual ~Person() {
        cout << "Person object destroyed\n";
    }
};
class Patient : public Person {
private:
    string disease;
    string department;
    bool emergency;
public:
    Patient(int i, string n, int a, string d, string dept, bool e)
        : Person(i, n, a), disease(d), department(dept), emergency(e) {}
};
```

```

void display() const override {
    cout << "\nPatient ID    : " << id;
    cout << "\nPatient Name  : " << name;
    cout << "\nAge          : " << age;
    cout << "\nDisease       : " << disease;
    cout << "\nDepartment    : " << department;
    cout << "\nEmergency     : " << (emergency ? "Yes" : "No") << endl;
}

bool isEmergency() const {
    return emergency;
}

~Patient() {
    cout << "Patient object destroyed\n";
}

};

class Doctor : public Person {
private:
    string specialization;
    string department;
public:
    Doctor(int i, string n, int a, string s, string d)
        : Person(i, n, a), specialization(s), department(d) {}

    void display() const override {
        cout << "\nDoctor ID    : " << id;
        cout << "\nDoctor Name  : " << name;
        cout << "\nAge          : " << age;
        cout << "\nSpecialization : " << specialization;
        cout << "\nDepartment    : " << department << endl;
    }

    ~Doctor() {

```

```

        cout << "Doctor object destroyed\n";
    }
};

class Billing {
private:
    double consultation;
    double treatment;
    double medicine;
public:
    Billing(double c, double t, double m)
        : consultation(c), treatment(t), medicine(m) {}
    double calculateBill() const {
        return consultation + treatment + medicine;
    }
    void displayBill() const {
        cout << "\n----- BILL DETAILS -----";
        cout << "\nConsultation : Rs." << consultation;
        cout << "\nTreatment   : Rs." << treatment;
        cout << "\nMedicine    : Rs." << medicine;
        cout << "\nTotal Bill  : Rs." << calculateBill() << endl;
    }
};

class Department {
private:
    string name;
    int beds;
    int doctors;
public:
    Department(string n, int b, int d)
        : name(n), beds(b), doctors(d) {}

```

```

void display() const {
    cout << "\nDepartment  : " << name;
    cout << "\nAvailable Beds: " << beds;
    cout << "\nDoctors      : " << doctors << endl;
}

string getName() const {
    return name;
}
};

class Hospital {
private:
    string hospitalName;
    vector<unique_ptr<Patient>> patients;
    vector<unique_ptr<Doctor>> doctors;
    vector<Department> departments;
public:
    Hospital(string name) : hospitalName(name) {}

    void addPatient(int id, string name, int age,
                    string disease, string dept, bool emergency) {
        patients.push_back(
            make_unique<Patient>(
                id, name, age, disease, dept, emergency
            )
        );
        cout << "\nPatient added successfully.\n";
    }

    void addDoctor(int id, string name, int age,
                    string specialization, string dept) {
        doctors.push_back(
            make_unique<Doctor>(

```



```

        id, name, age, specialization, dept
    )
);
cout << "\nDoctor added successfully.\n";
}

void addDepartment(string name, int beds, int doctors) {
    departments.emplace_back(name, beds, doctors);
    cout << "\nDepartment added successfully.\n";
}

void displayPatients() const {
    cout << "\n===== PATIENT DETAILS =====\n";

    if (patients.empty()) {
        cout << "No patients available.\n";
        return;
    }

    for (const auto& p : patients)
        p->display();
}

void displayDoctors() const {
    cout << "\n===== DOCTOR DETAILS =====\n";

    if (doctors.empty()) {
        cout << "No doctors available.\n";
        return;
    }

    for (const auto& d : doctors)
        d->display();
}

void displayDepartments() const {

```

```

    cout << "\n===== DEPARTMENTS =====\n";

    for (const auto& d : departments)
        d.display();
}

void emergencyPatients() const {
    cout << "\n===== EMERGENCY PATIENTS =====\n";
    bool found = false;
    for (const auto& p : patients) {
        if (p->isEmergency()) {
            p->display();
            found = true;
        }
    }
    if (!found)
        cout << "No emergency patients.\n";
}

void searchPatient(int id) const {
    for (const auto& p : patients) {
        if (p->getId() == id) {
            cout << "\nPatient found:\n";
            p->display();
            return;
        }
    }
    cout << "\nPatient not found.\n";
}

};

int main() {
    Hospital hospital("City Care Hospital");

```

```

int choice;

do {
    cout << "\n\n===== ";
    cout << "\n    HOSPITAL MANAGEMENT SYSTEM";
    cout << "\n===== ";
    cout << "\n1. Add Department";
    cout << "\n2. Add Doctor";
    cout << "\n3. Add Patient";
    cout << "\n4. Display Departments";
    cout << "\n5. Display Doctors";
    cout << "\n6. Display Patients";
    cout << "\n7. Display Emergency Patients";
    cout << "\n8. Search Patient";
    cout << "\n9. Generate Bill";
    cout << "\n0. Exit";
    cout << "\nEnter choice: ";
    cin >> choice;
    if (choice == 1) {
        string name;
        int beds, doctors;

        cout << "Enter department name: ";
        cin >> name;
        cout << "Enter number of beds: ";
        cin >> beds;
        cout << "Enter number of doctors: ";
        cin >> doctors;
        hospital.addDepartment(name, beds, doctors);
    }
    else if (choice == 2) {

```

```

    int id, age;
    string name, specialization, dept;
    cout << "Enter doctor ID: ";
    cin >> id;
    cout << "Enter doctor name: ";
    cin >> name;
    cout << "Enter age: ";
    cin >> age;
    cout << "Enter specialization: ";
    cin >> specialization;
    cout << "Enter department: ";
    cin >> dept;
    hospital.addDoctor(
        id, name, age,
        specialization, dept
    );
}
else if (choice == 3) {
    int id, age;
    string name, disease, dept;
    char emergency;
    cout << "Enter patient ID: ";
    cin >> id;
    cout << "Enter patient name: ";
    cin >> name;
    cout << "Enter age: ";
    cin >> age;
    cout << "Enter disease: ";
    cin >> disease;
    cout << "Enter department: ";

```

```

    cin >> dept;

    cout << "Emergency patient? (Y/N): ";

    cin >> emergency;

    bool e = (emergency == 'Y' || emergency == 'y');

    hospital.addPatient(
        id, name, age,
        disease, dept, e
    );
}

else if (choice == 4) {
    hospital.displayDepartments();
}

else if (choice == 5) {
    hospital.displayDoctors();
}

else if (choice == 6) {
    hospital.displayPatients();
}

else if (choice == 7) {
    hospital.emergencyPatients();
}

else if (choice == 8) {
    int id;

    cout << "Enter patient ID to search: ";

    cin >> id;

    hospital.searchPatient(id);
}

else if (Choice == 9) {
    double consultation, treatment, medicine;

    cout << "Enter consultation fee: ";

```

```

        cin >> consultation;

        cout << "Enter treatment fee: ";

        cin >> treatment;

        cout << "Enter medicine fee: ";

        cin >> medicine;

        Billing bill(
            consultation,
            treatment,
            medicine );

        bill.displayBill();

    }

    else if (choice == 0) {

        cout << "\nExiting Hospital Management System...\n";

    }

    else {

        cout << "\nInvalid choice!\n";

    }

} while (choice != 0);

return 0;

}

```

OUTPUT:

main.cpp	Run	Output
<pre> int main() { do { else if (choice == 6) { hospital.displayPatients(); } else if (choice == 7) { hospital.emergencyPatients(); } else if (choice == 8) { int id; cout << "Enter patient ID to search' "; cin >> id; hospital.searchPatient(id); } else if (choice == 9) { double consultation, treatment, medicine; cout << "Enter consultation fee: "; cin >> consultation; cout << "Enter treatment fee: "; cin >> treatment; cout << "Enter medicine fee: "; cin >> medicine; Billing bill(consultation, treatment, medicine); } } while (choice != 0); return 0; } </pre>		<pre> 1. Add Department 2. Add Doctor 3. Add Patient 4. Display Departments 5. Display Doctors 6. Display Patients 7. Display Emergency Patients 8. Search Patient 9. Generate Bill 0. Exit Enter choice: 1 Enter department name: cardiology Enter number of beds: 20 Enter number of doctors: 5 Department added successfully. ===== HOSPITAL MANAGEMENT SYSTEM ===== 1. Add Department 2. Add Doctor 3. Add Patient 4. Display Departments 5. Display Doctors 6. Display Patients 7. Display Emergency Patients 8. Search Patient 9. Generate Bill 0. Exit Enter choice: 2 Enter doctor ID: 101 Enter doctor name: Iumar Enter age: 45 Enter specialization: cardio Enter department: cardiology Doctor added successfully. </pre>

```
main.cpp
1 int main() {
2     do {
3
4         else if (choice == 6) {
5             hospital.displayPatients();
6         }
7
8         else if (choice == 7) {
9             hospital.emergencyPatients();
10        }
11
12        else if (choice == 8) {
13            int id;
14            cout << "Enter patient ID to search: ";
15            cin >> id;
16            hospital.searchPatient(id);
17        }
18
19        else if (choice == 9) {
20            double consultation, treatment, medicine;
21            cout << "Enter consultation fee: ";
22            cin >> consultation;
23            cout << "Enter treatment fee: ";
24            cin >> treatment;
25            cout << "Enter medicine fee: ";
26            cin >> medicine;
27
28            Billing bill(
29                consultation,
30                treatment,
31                medicine
32            );
33            bill.displayBill();
34        }
35
36        else if (choice == 0) {
37            cout << "\nExiting Hospital Management System...\n";
38        }
39
40        else {
41            cout << "Invalid choice!\n";
42        }
43    } while (choice != 0);
44}
```

Run

Output

```
=====
HOSPITAL MANAGEMENT SYSTEM
=====
1. Add Department
2. Add Doctor
3. Add Patient
4. Display Departments
5. Display Doctors
6. Display Patients
7. Display Emergency Patients
8. Search Patient
9. Generate Bill
0. Exit
Enter choice: 3
Enter patient ID: 201
Enter patient name: RAVI
Enter age: 35
Enter disease: HEART
Enter department: CARDIOLOGY
Emergency patient? (Y/N): Y
Patient added successfully.

=====
HOSPITAL MANAGEMENT SYSTEM
=====
1. Add Department
2. Add Doctor
3. Add Patient
4. Display Departments
5. Display Doctors
6. Display Patients
7. Display Emergency Patients
8. Search Patient
9. Generate Bill
0. Exit
Enter choice: 6

===== PATIENT DETAILS =====
Patient ID      : 201
Patient Name    : RAVI
Age             : 35
Disease         : HEART
Department      : CARDIOLOGY
Emergency       : Yes
```

```
else if (choice == 8) {
    int id;
    cout << "Enter patient ID to search: ";
    cin >> id;
    hospital.searchPatient(id);
}

else if (choice == 9) {
    double consultation, treatment, medicine;
    cout << "Enter consultation fee: ";
    cin >> consultation;
    cout << "Enter treatment fee: ";
    cin >> treatment;
    cout << "Enter medicine fee: ";
    cin >> medicine;

    Billing bill(
        consultation,
        treatment,
        medicine
    );
    bill.displayBill();
}

else if (choice == 0) {
    cout << "\nExiting Hospital Management System...\n";
}

else {
    cout << "Invalid choice!\n";
}
```

```
=====
HOSPITAL MANAGEMENT SYSTEM
=====
1. Add Department
2. Add Doctor
3. Add Patient
4. Display Departments
5. Display Doctors
6. Display Patients
7. Display Emergency Patients
8. Search Patient
9. Generate Bill
0. Exit
Enter choice: 9
Enter consultation fee: 500
Enter treatment fee: 300
Enter medicine fee: 1500

----- BILL DETAILS -----
Consultation : Rs.500
Treatment    : Rs.300
Medicine     : Rs.1500
Total Bill   : Rs.2300

=====
HOSPITAL MANAGEMENT SYSTEM
=====
1. Add Department
2. Add Doctor
3. Add Patient
4. Display Departments
5. Display Doctors
6. Display Patients
7. Display Emergency Patients
8. Search Patient
9. Generate Bill
0. Exit
```